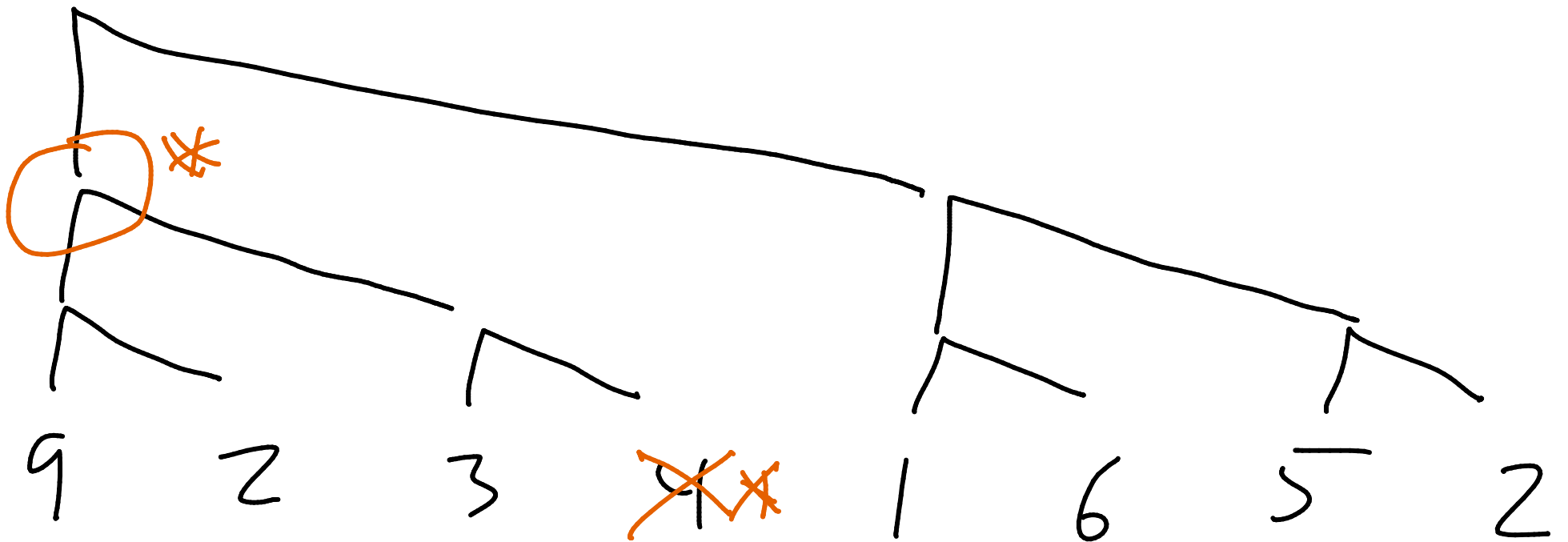


Today

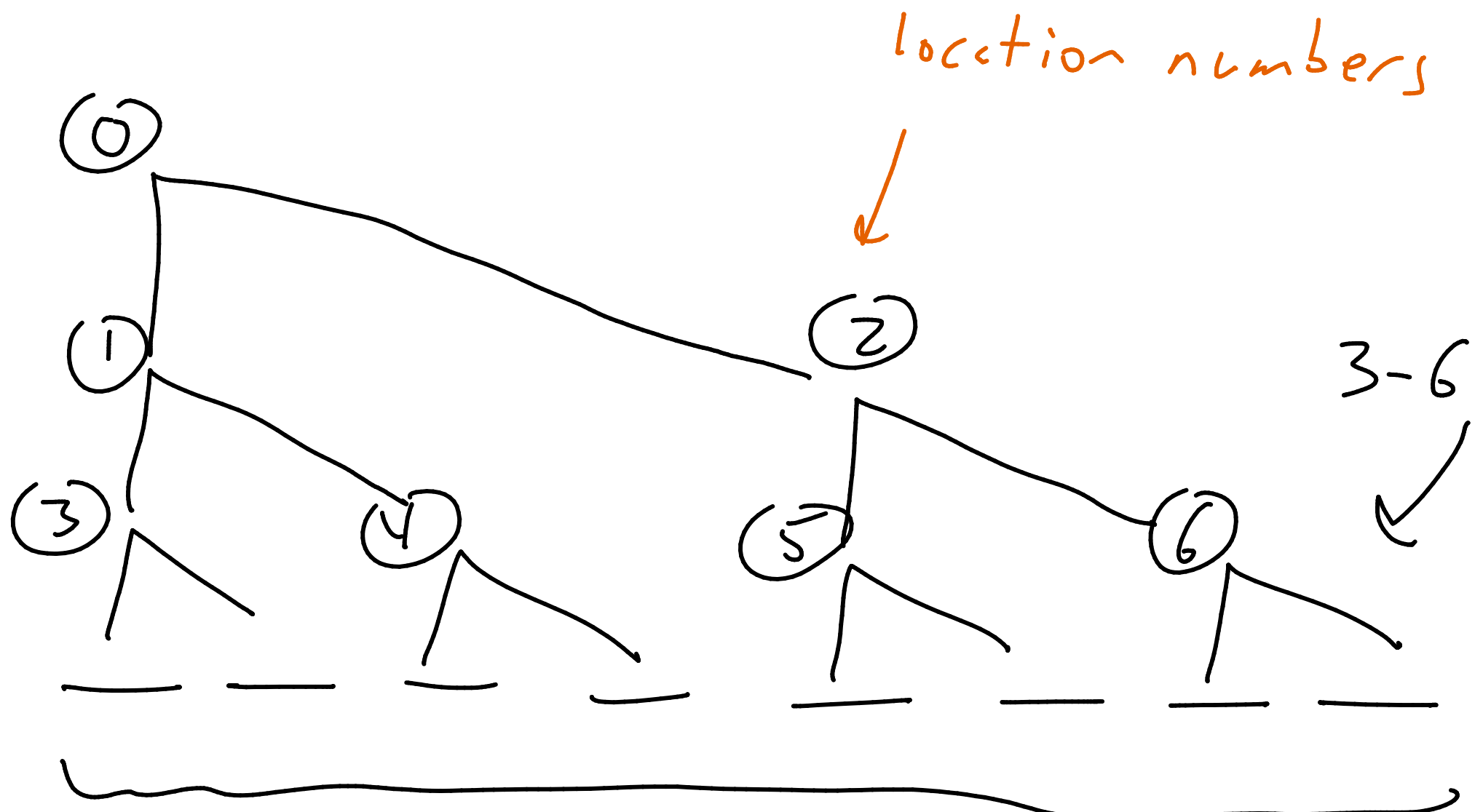
- wrapup on pre-scan
 - filter/pack
 - quicksort
 - mergesort
-

Pre scan - don't need tree at all, just do it all in place in an array



every step of the way, value of a node can just be stored in right most child at leaf level

How do you find that spot?



you know at start how tall tree
is because you know size of array

At any location i , the children
are $2i+1$ and $2i+2$.

To get to the bottom.

At location i ,
right child is $2i+2$

its two children are $2(2i+2)+1$

right grandchild $\rightarrow 2(2i+2)+2$

To get to bottom level

- write a loop (slow and not bracing)
- just work out the formula based on the # of levels in between

Pre-scan (sum of values to the left)

has work $O(n)$

scan $O(\log n)$

exponential parallelism $O(n/\log n)$

Parallel pack/filter

Find all names in a list with
a starting letter $\leq j$

dave liz evan rae jessie mitch



dave evan jessie

In parallel, test each in original list
and in a new list put in a 1 if
satisfies query, and a 0 if not

dave liz evan rae jessie mitch

1	0	1	0	1	0
Calculate pre scan on this (sum of values to left)					
0	1	1	2	2	3

To construct answer, look at the 1s
in the first array, then the
location where it goes is the
matching spot in the second

dave evan jessie

0 1 2

① Identify the 1's
Work $O(n)$
Span $O(1)$

② Pre-scan
Work $O(n)$
Span $O(\log n)$

③ Copy the values of 1s into locs
Work $O(n)$ $\leftarrow$ look at each item. If 1, copy value.

Span $O(1)$ $\leftarrow$ do this in parallel for each item, there's no chain of steps

① + ② + ③

Work $O(n)$

Span $O(\log n)$

Parallelism $O(n/\log n)$

Quicksort

8 2 15 3 9 1 7 6

Pick one item = "pivot"

get all items $\leq$ pivot to its left

> pivot to its right

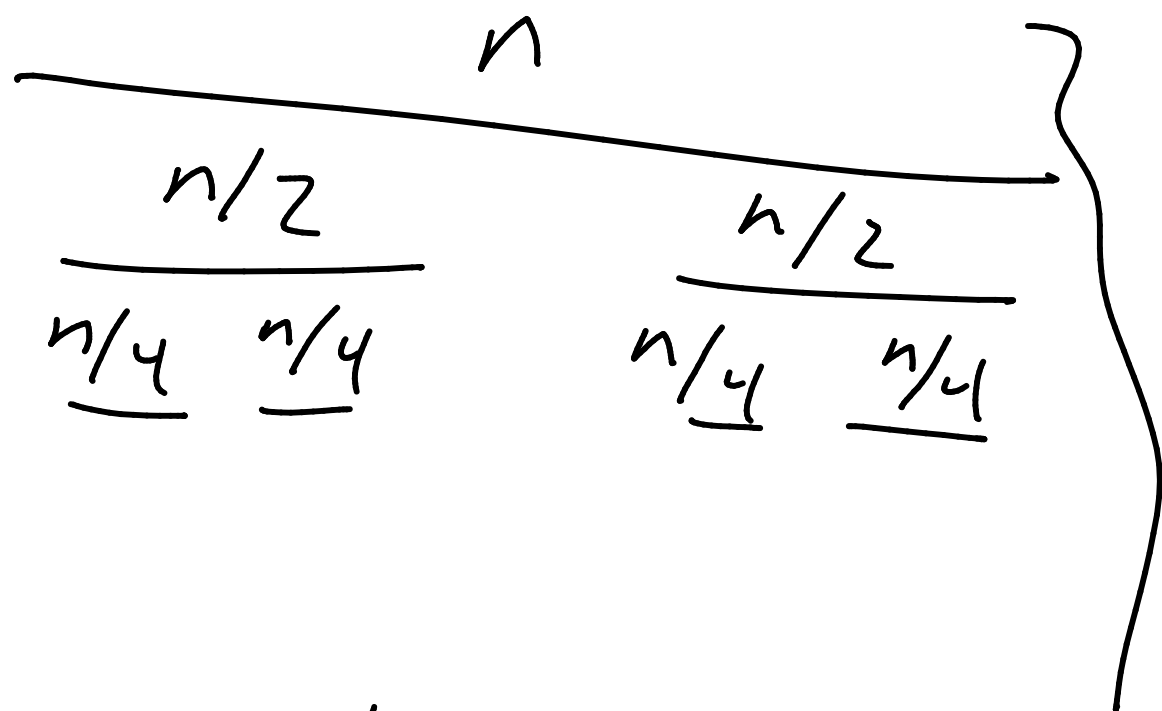
algorithm, in $O(n)$ (CS 201) does
this for getting items in place around the
pivot

get something like

2 1 3 6 7 8 9 15

$\nwarrow$ $\nearrow$

Quick sort each of these



Assuming even splits which is not a given assumption

$\log n$ levels

n work on each level

Total work is usually $O(n \log n)$

[Worst case, when all items land on one side of the pivot, at every level, is $O(n^2)$]

Fork join part is easy, but pivoting step as done classically is all sequential

Instead, to "pivot"

(set items $\leq$ pivot on left
 $>$ pivot on right)

This is just two filter operations

8 2 15 3 9 1 7 6

filter all ≤ 8 {
filter all > 8 }

work $O(n)$

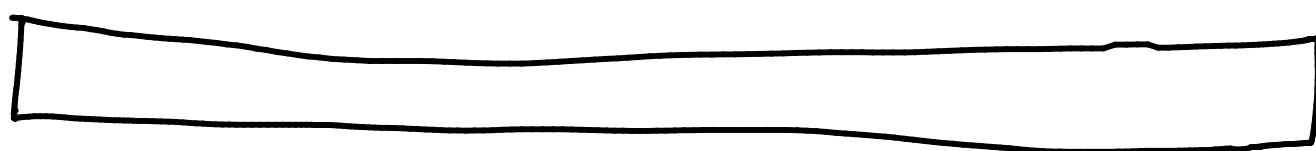
span $O(\log n)$

At first level

work $\approx n$

how many levels? $\approx \log n$

~~How much~~ work over all levels $\approx n \log n$

 n work


 $\approx n/2$ work $\approx n/2$ work $\approx n$ work

 $\approx n$ work

w/ $\approx \log n$ levels, total
work $\approx n \log n$

First level, span $\approx \log n$

Each filter has span $\approx \log n$
there are 2, but not dependant on
each other, could be done at same time

What about second level?

Span $\approx \log\left(\frac{n}{2}\right)$

Total Span

$\approx \log(n) + \log\left(\frac{n}{2}\right) + \log\left(\frac{n}{4}\right) + \dots + 1$

upper bound
 $\approx$
 $\log n * \log n$
of levels = $\log n$

Whole algorithm

$$\text{work} = O(n \log n)$$

assuming not
worst case or
-ish

$$\text{span} = O((\log n)^2)$$

$$\text{parallelism} = O\left(\frac{n \log n}{(\log n)^2}\right)$$

$$= O(n / \log n)$$

exponential speedup

Mergesort

8 2 15 3

mergesort

2 3 8 15

merge together

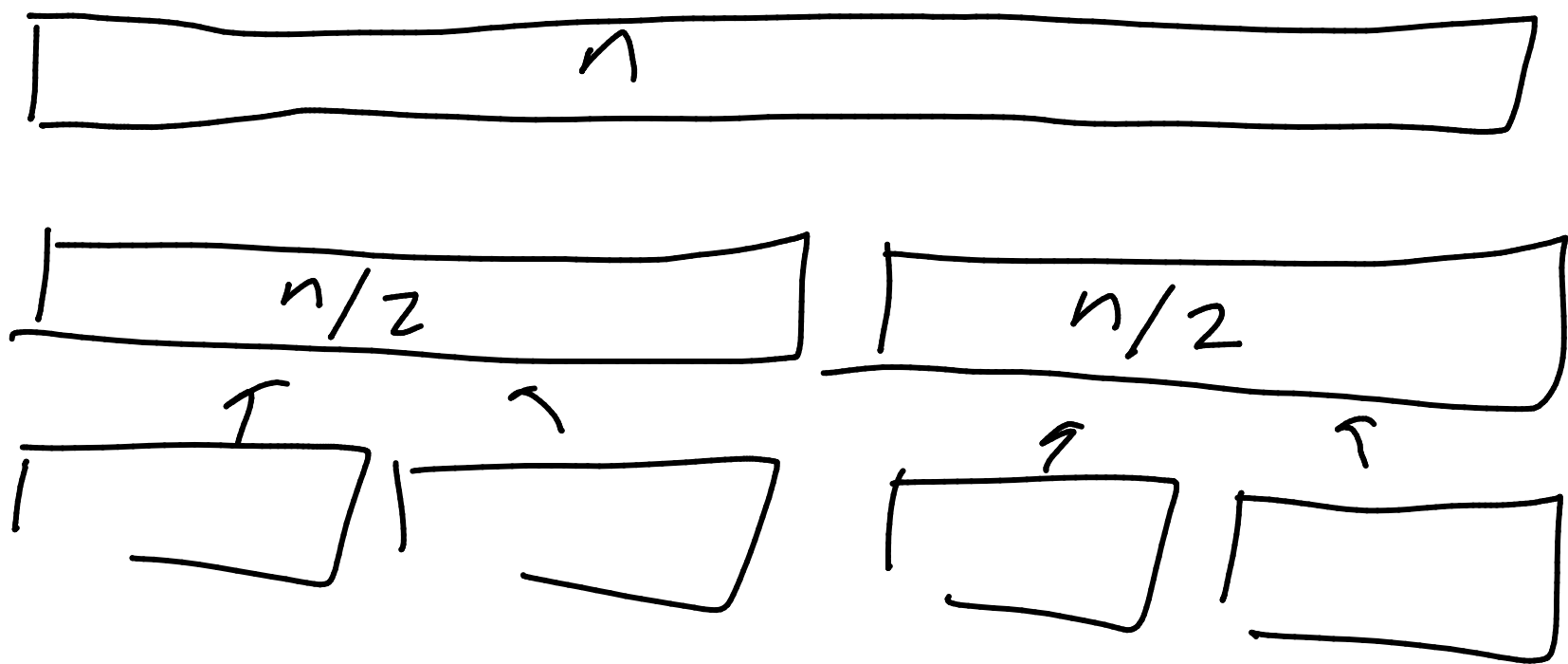
9 1 7 6

mergesort

1 6 7 9



What happens if you do forkjoin
parallelism for the recursive mergesorts,
but at each level the merge
remains sequential?



work:

- top level: merge $\approx n$

- next level: $\approx n/2$ for each half

$$\frac{n}{2} + \frac{n}{2} = n$$

$$\text{next level} \approx \frac{n}{4} + \frac{n}{4} + \frac{n}{4} + \frac{n}{4} = n$$

w/ $\log n$ levels,

$$\text{work} = O(n \log n)$$

At top level, span $\approx n$

Second level, span $\approx n/2$

in total

$$\text{span} \approx \underbrace{n + \frac{n}{2} + \frac{n}{4} + \dots + 1}_{\log n \text{ terms}} = 2n$$

$$\text{Span} \approx \cancel{n \log n} = 2n$$

ugh, bad

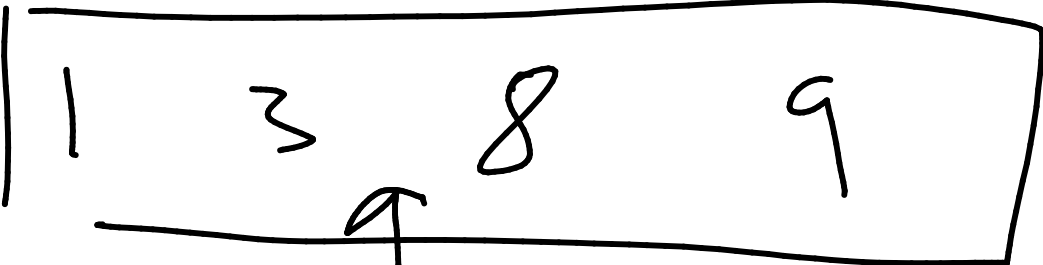
~~Speedup~~

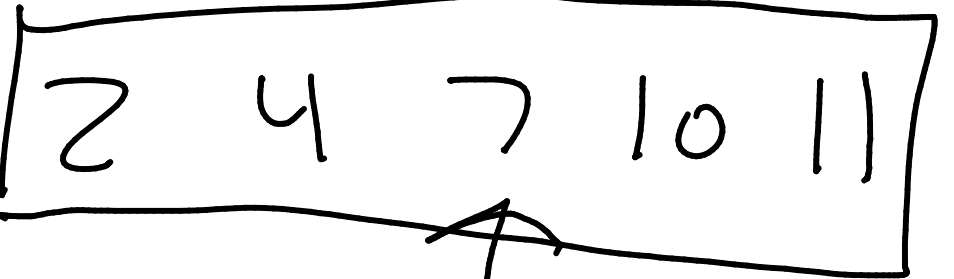
$$\text{Parallelism} = O\left(\frac{n \log n}{2n}\right) = O(\log n)$$

How do you merge better?


(1) Find the median of ^{bigger} ~~one~~ half
 $10(1)$ because already sorted

(2) Binary search on other half to
find location where all items to
left $\leq$ median, to right
 $>$ median $O(\log n)$





binary search

(3) Recursively do this process on
the left portions of each

(4) $\dots$
 $\dots$ on the right portions of
each

Analysis is tricky

$$\text{speedup} = O\left(\frac{n}{(\log n)^2}\right)$$